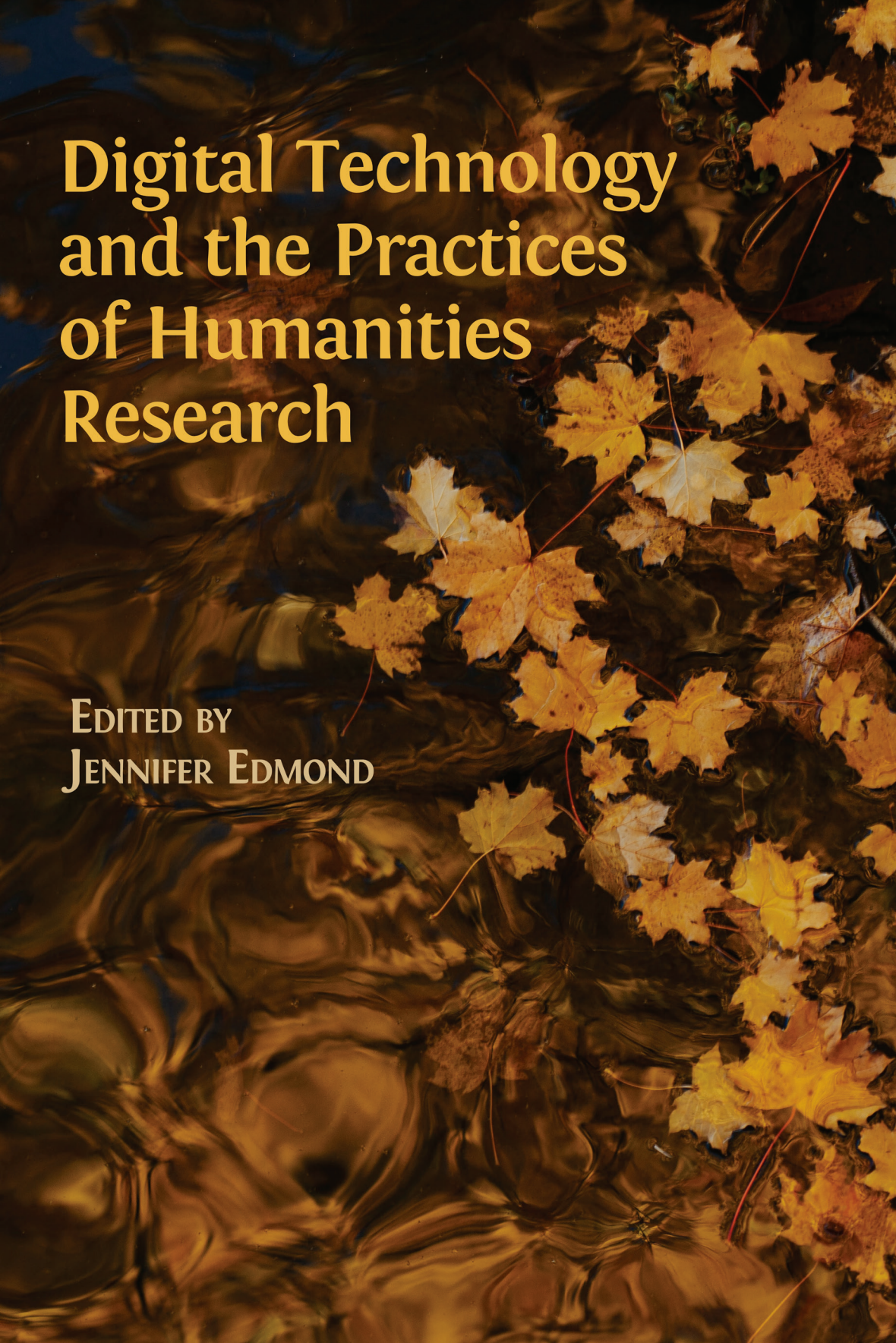
Digital Technology and the Practices of Humanities Research
Edited by
Jennifer Edmond

https://www.openbookpublishers.com

ISBN Paperback: 978-1-78374-839-6

ISBN Hardback: 978-1-78374-840-2

ISBN Digital (PDF): 978-1-78374-841-9

ISBN Digital ebook (epub): 978-1-78374-842-6

ISBN Digital ebook (mobi): 978-1-78374-843-3

ISBN Digital (XML): 978-1-78374-844-0

DOI: 10.11647/OBP.0192

Cover image: photo by Nanda Green on Unsplash https://unsplash.com/photos/BeVWHMXYwwo

Cover design: Anna Gatti

6. 'Black Boxes' and True Colour — A Rhetoric of Scholarly Code

Joris J. van Zundert, Smiljana Antonijević, and Tara L. Andrews

Introduction

Software pervades society. As Lev Manovich, Steven Jones, and David Berry have shown, there is hardly any form of contemporary data or information that has not been touched by digital means at some point during its creation.[1] The humanities, whose scholars study the data and information that is connected to social, historical, and cultural artefacts, are affected by a similar pervasiveness of software.[2] Programmers write software in a form of text known as source code: a series of instructions for how to perform a task, or a set of tasks, that the computer carries out. As software pervades the humanities, so its source code increasingly forms part of the makeup of the method and design in research projects in the humanities fields; this

1 Lev Manovich, *Software Takes Command: Extending the Language of New Media*, International Texts in Critical Media Aesthetics 5 (New York, NY: Bloomsbury Academic, 2013); Steven E. Jones, *The Emergence of the Digital Humanities* (New York, NY: Routledge, 2014); David M. Berry, *Critical Theory and the Digital*, Critical Theory and Contemporary Society (New York, NY: Bloomsbury Academic, 2014), https://doi.org/10.5040/9781501302114

2 Cf. Jones, *Emergence of the Digital Humanities*; Manovich, *Software Takes Command*.

 https://doi.org/10.11647/OBP.0192.06

is the particular focus of the emerging discipline, or methodology, or movement, known as digital humanities (DH).

As the expressions of a *technē* whose inner workings are opaque to most humanities scholars, code and codework[3] are all too often treated as invisible hands, which influence humanities research in ways that are neither transparent nor accounted for. The software used in research is treated as a 'black box' in the sense of information science — that is, it is expected to produce a certain output given a certain input — but, at the same time, it is often mistrusted precisely for this same lack of transparency. It is also often perceived as a mathematical — and thus value-neutral and socially inert — instrument; moreover, these two seemingly contradictory perceptions need not be mutually exclusive.

The lack of knowledge about what is actually taking place in these software 'black boxes' and about how they are made introduces serious problems for evaluation and trust in humanities research. If we cannot read code or see the workings of the software as it functions, we can experience it only in terms of its interface and its output, neither of which seem subject to our control. Yet, code is written by people, thus making it a social construct that embeds and expresses social and ideological beliefs of which it is — intentionally or not, directly or as a side effect — an agent.[4] Code is a more or less a withdrawn or even covert, but non-neutral, technology.[5] Therefore, when humanities scholars use software, they may unwittingly import certain methodological and epistemological assumptions inherent in that software into their research fields. Moreover, the invisibility and un-critiqued use of code in the humanities means that the scholarly quality and contribution of codework goes both uncredited and unaccounted for. To mitigate problems with academic evaluation and credit, a much greater insight into code and codework in the humanities

3 We understand 'codework' to mean all the work involved in creating software source code that is more than just the act of writing the code. As we will explain further on, it encompasses many concrete and cognitive scholarly tasks. We use 'codework' as a broadly inclusive term, while we use 'coding' more narrowly as the act of writing source code.

4 Tara McPherson, 'Why Are the Digital Humanities So White? Or Thinking the Histories of Race and Computation', in *Debates in the Digital Humanities*, ed. by Matthew K. Gold (Minneapolis: University of Minnesota Press, 2012), pp. 139–60, https://doi.org/10.5749/minnesota/9780816677948.003.0017, http://dhdebates.gc.cuny.edu/debates/text/29

5 David M. Berry, *The Philosophy of Software: Code and Mediation in the Digital Age* (Basingstoke: Palgrave Macmillan, 2011).

is urgently required by those who engage in such evaluation; for instance, how coders approach their tasks, what decisions go into its production, and how code interacts with its environment. The purpose of this chapter is to provide some of that insight in the form of an ethnography of codework, wherein we observe the decisions that programmers make, and how they understand their own activities. This 'studying-up'[6] of people who hold epistemological and methodological power — in this case coding power — follows in the footsteps of ethnographies of technoscientific practice[7] and reflections on coding and tool development in DH.[8] Like other ethnographic studies, our small-scale exploration does not aspire to be fully representative of DH codework, but to initiate a debate about some still overlooked elements of this practice. We conclude this chapter with a discussion about our findings and several recommendations for how codework should be approached by programmers, scholars, and administrators in the humanities.

Background

Code can be understood as an argument in a way that is congruent with Alan Galey and Stan Ruecker's understanding of the epistemological status of graphical user interfaces as argument.[9] Code and codework

6 Laura Nader, 'Up the Anthropologist: Perspectives Gained from Studying Up', in *Reinventing Anthropology*, ed. by D. H. Hymes, Ann Arbor Paperbacks (University of Michigan Press, 1972), pp. 284–311, http://www.dourish.com/classes/readings/Nader-StudyingUp.pdf

7 E. Gabriella Coleman, *Coding Freedom: The Ethics and Aesthetics of Hacking* (Princeton (US), Woodstock (UK): Princeton University Press, 2013), http://gabriellacoleman.org/Coleman-Coding-Freedom.pdf; G. Coleman, *Hacker, Hoaxer, Whistleblower, Spy: The Many Faces of Anonymous* (London, New York: Verso, 2014); D. Forsythe, and D. J. Hess, *Studying Those Who Study Us: An Anthropologist in the World of Artificial Intelligence* (Stanford, CA: Stanford University Press, 2001).

8 Stephen Ramsay and Geoffrey Rockwell, 'Developing Things: Notes toward an Epistemology of Building in the Digital Humanities', in *Debates in the Digital Humanities*, ed. by Matthew K. Gold (Minneapolis: University of Minnesota Press, 2012), pp. 75–84, https://doi.org/10.5749/minnesota/9780816677948.003.0010, http://dhdebates.gc.cuny.edu/debates/text/11; Susan Schreibman and Ann M. Hanlon, 'Determining Value for Digital Humanities Tools: Report on a Survey of Tool Developers', *Digital Humanities Quarterly*, 4.2 (2010), http://digitalhumanities.org/dhq/vol/4/2/000083/000083.html; Nikolai Bezroukov, 'Open Source Software Development as a Special Type of Academic Research: Critique of Vulgar Raymondism', *First Monday*, 4.10 (1999), https://doi.org/10.5210/fm.v4i10.696

9 Alan Galey and Stan Ruecker, 'How a Prototype Argues', *Literary and Linguistic Computing*, 25.4 (2010), 405–24, https://doi.org/10.1093/llc/fqq021

share many properties with text and writing, indeed many more than most programmers and scholars usually acknowledge. When a programmer writes software, the result is not merely a digital object with a specific computational function. It is a program that can be executed by a computer, but, as so-called source code, it is also a text readable by humans (primarily, but not exclusively, programmers).[10] In the case of codework in humanities research, this text is also a part of an encompassing and larger epistemological framework comprising research design, theory, activities, interactions, and outputs. In the digital humanities context, the code part of this framework arises from a combination of the programmer's technical skills, her theoretical background knowledge (concerning both the humanities topic and computational modelling), and interpretations of the conversations she has had with collaborators, both academic and technical. It follows that, from an epistemic point of view, the practice of the programmer is no different from the practice of the scholar when it comes to writing.[11] Both are creating theories about existing epistemic objects (e.g. text and material artefacts, or data) by developing new epistemic objects (e.g. journal articles and critical editions, or code) to formulate and support these theories. In this sense, our view connects back to Bernard Cerquiglini's position that the scholarly editions of texts are not mere re-representations of some existing textual content, but *theories* about that content.[12]

The analogy we draw between code and programmers, on the one hand, and print publications and scholars, on the other, parallels Bruno Latour's comparison of machines and engineers, with texts and writers.[13] In relation to the practice of developing machines, and their application in scientific research, Latour also makes reference to the

10 Moritz Hiller, 'Signs o' the Times: The Software of Philology and a Philology of Software', *Digital Culture and Society*, 1.1 (2015), 152–63, https://doi.org/10.14361/dcs-2015-0110

11 Joris J. van Zundert, 'Author, Editor, Engineer: Code & the Rewriting of Authorship in Scholarly Editing', *Interdisciplinary Science Reviews*, 40.4 (2016), 349–75, https://doi.org/10.1080/03080188.2016.1165453

12 Bernard Cerquiglini, *In Praise of the Variant: A Critical History of Philology* (Baltimore, MD: The Johns Hopkins University Press, 1999).

13 Bruno Latour, 'Where Are the Missing Masses, Sociology of a Few Mundane Artefacts', in *Shaping Technology-Building Society. Studies in Sociotechnical Change*, ed. by Wiebe Bijker and John Law (Cambridge, MA: MIT Press, 1992), pp. 225–59, http://www.bruno-latour.fr/node/258

idea of the 'black box', which he defines as any technology, instrument, theory, or algorithm that is considered to be so well established as fact that it is beyond question; scientific controversies surrounding the construction of the 'black box' have arisen, been resolved, and become effectively invisible.[14] In Latour's explanation of science as a social act, the construction of 'black boxes' allows larger epistemological constructs to develop; by the same token, controversies in science can be understood as attempts to construct and defend, or attack and destroy, particular 'black boxes' in the making. A 'black box' thus comes into being precisely through the establishment of trust in its correct functioning, which is done by seeking a consensus about its correctness within the bounds, and according to the social mechanisms, of the scientific community.

In the humanities, however, the term 'black box' is often used to signal some unknown: a theory or instrument that has not undergone critical inspection and cannot, therefore, be trusted. Thus the labelling of a particular software technology as a 'black box' has come to mean, in some parts of the humanities, precisely the opposite of what was intended: rather than signalling that 'this is a trusted instrument', it signals 'this is an instrument which is suspect, and deserving of critical attention.'[15] Arguably the perverse implications of the label, and the

14 Bruno Latour, *Science in Action: How to Follow Scientists and Engineers Through Society* (Cambridge, MA: Harvard University Press, 1988).

15 See, for instance, Max Kemman, Martijn Kleppe, and Stef Scagliola, 'Just Google It: Digital Research Practices of Humanities Scholars', in *Proceedings of the Digital Humanities Congress 2012*, ed. by Clare Mills, Michael Pidd, and Esther Ward (Sheffield: HRI Online Publications, 2014), http://www.hrionline.ac.uk/openbook/chapter/dhc2012-kemman: 'Google introduces a black box into the digital research practices of scholars, but interestingly enough this does not seem to influence the trust of the majority of scholars in search results'; also, P. Svensson, *Big Digital Humanities: Imagining a Meeting Place for the Humanities and the Digital* (Ann Arbor, MI: University of Michigan Press, 2016), https://doi.org/10.1353/book.52252, http://hdl.handle.net/2027/spo.13607060.0001.001, talks about 'the importance of providing material results to the users rather than quantitative "black boxes" results' (p. 92). Svensson, interestingly, also uses the metaphor for the organisational mechanisms of the globally overarching organisation for digital humanities, ADHO (Svensson, *Big Digital Humanities*, p. 79). Johanna Drucker, although not specifically using the metaphor of 'black box', talks about 'reification of misinformation' when addressing computational quantitative measures on data we cannot see, with provenance we cannot verify, using algorithms we do not know (Johanna Drucker, 'Should Humanists Visualize Knowledge?', *Vimeo*, video lecture at Lehigh University, Bethlehem, Pennsylvania, 2016, https://vimeo.com/140307034).

suspicion with which so-called 'black boxes' are treated, are precisely the symptoms of the failure of the existing social, scholarly mechanisms, within those sectors of the humanities that are most distant from the empirical end of the science spectrum, to incorporate instruments and theories that arise from without. The result is a poignant mutual incomprehension: those who create software often understand their goal precisely to be the construction of a (trustworthy) 'black box', and they draw upon the mechanisms of science to do so — for what programmer wishes her code to be considered untrustworthy? And yet this very attempt to build the trust necessary for the instrument to attain 'black box' status, especially if the attempt is accompanied by the sort of discourse common in the empirical sciences, causes distrust in a community where consensus and dissent work differently.

Put another way: the very qualities and practices that, in other contexts, would create trust in software tools, now tend to diminish trust in them in the humanities context. In order to begin to counteract this paradox we can perhaps draw on the idea of code as an argument. As Richard Coyne and David Berry, among others, have shown, the internal structure and narrative of code ought not to be regarded as a mathematically infallible epistemological construct, although formal and mathematical logic is involved in its composition, just as logic has a natural place within rhetoric.[16] If we consider code as a rhetorical rather than a mathematical argument, it parallels humanities knowledge production in terms of theory and methodology. Code can thus inherit the multi-perspective, problematising nature and diverse styles of reasoning that are particular marks of methodology in the humanities. From this perspective, different code bases represent different theories, each of which needs to show its distinctive, true colours in order to be adequately recognised and evaluated.

Until now, however, most fields within the humanities lack a system for approaching, in a critical fashion, any argument that code presents, and for evaluating the workings of software. The discourse critiquing and evaluating code in the (digital) humanities has mostly focused on tenure track evaluation and peer review of the 'surface' of digital

16 Richard Coyne, *Designing Information Technology in the Postmodern Age: From Method to Metaphor*, A Leonardo Book (Cambridge, MA: MIT Press, 1995); Berry, *Critical Theory*.

objects, i.e. the resulting interface or visual presentation.[17] Within the humanities, very little work has been done on practical code review or on the evaluation of the inner logic of code.[18] To this end, some work has been done in new media and software studies, especially where software has a role as a production tool of cultural artefacts in film, art, and so forth.[19] This work, however, primarily concerns itself with the 'theoretical discussion of how software interacts with society, influencing our perception of the world'.[20] With some noted exceptions,[21] academic journal articles in the humanities rarely engage with the actual source code that underlies computationally-derived research results. A methodological examination essentially restricts itself to the results obtained from a graphical interface, or the interpretation of the quantitative results generated by a software program. A typical paper might report what statistical measure had been used, but generally omits to mention which software was used to make the measurement; in the case of project-specific software, the quality of its implementation is not examined. Many of the standard mechanisms for quality control in the software industry, such as line-by-line code review, unit testing, regression testing, and measurement of the extent to which the tests are comprehensive ('code coverage'), are routinely omitted in the humanities programming context, including in larger projects and even some centres.[22] Yet it is this type of engineering knowledge (crucial as it

17 Susan Schreibman, Laura Mandell, and Stephen Olsen, 'Introduction', *Profession* (2011), 123–201, https://doi.org/10.1632/prof.2011.2011.1.123; Kathleen Fitzpatrick, 'Peer Review, Judgment, and Reading', *Profession* (2011), 196–201, https://doi.org/prof.2011.2011.1.196

18 Joris J. van Zundert and Ronald Haentjens Dekker, 'Code, Scholarship, and Criticism: When Is Coding Scholarship and When Is It Not?', *Digital Scholarship in the Humanities* (2017), https://doi.org/10.1093/llc/fqx006

19 E.g., Manovich, *Software Takes Command*; and Mark C. Marino, 'Field Report for Critical Code Studies, 2014', *Computational Culture*, 4 (2014), http://computationalculture.net/article/field-report-for-critical-code-studies-2014%E2%80%A8

20 Chiara Bernardi, 'Working Towards a Definition of the Philosophy of Software', *Computational Culture*, 2 (2012), http://computationalculture.net/review/working-towards-a-definition-of-the-philosophy-of-software

21 For instance, *Cultural Analytics*, https://culturalanalytics.org, and *Computational Culture*, http://computationalculture.net/ could be mentioned. However, even in issues of these publication platforms, which are geared specifically towards critical engagement with data and software, one searches in vain for actual source code criticism.

22 Although we do not wish to call out specific examples of projects or tools that omit these practices, because the problem is so widespread, the reader is invited to

is in establishing the correct working of the code and gauging its inbuilt assumptions) that would be fundamental to the critical examination of the software that is applied in humanities research. Perhaps the fact that software and code peer review do not count towards academic credit[23] in most academic contexts plays into this state of affairs. There is no incentive for humanities researchers to consider the scientific or technical quality of the software tools they wield, nor is there sufficient training to acquire the skills to do so. Software engineering in the humanities ranges from professional teams working in conformance with industry testing best practice, to untested one-off scripts created by individuals. The scholars who rely on these tools lack the means to gauge the quality of either.

As code is an increasingly important epistemic object in humanities research, the state of affairs described above creates a real methodological problem; this gives rise to an urgent need for a practical examination and theoretical discussion of how software reflexively interacts with humanities research. We contend that both code as an epistemic object, and codework as an epistemic practice, must be given proper theoretical and methodological recognition in the digital humanities, along with the consequences and the rewards that such recognition bears. The current practice of 'black-boxing' the code results in a neglect of its epistemological contributions, and imperils one of the key components of knowledge production in the digital humanities.

There are three steps in particular that could be taken towards solving the deficiencies in current peer review practices concerning code and codework. First, there is a need for peer review and the critical examination of source code itself.[24] Second, open publishing of code in verifiable ways is already easily facilitated through existing public repositories such as GitHub and SourceForge, or institutionally-

peruse the code bases of those tools and projects that have been made open source, and to reflect on the fact that quite a bit of software in the humanities is not open source at all. The authors have frequently heard 'I would be embarrassed for others to see the code' cited as a reason for keeping source code in humanities projects closed.

23 Cf. again Schreibman, Mandell, and Olsen, 'Introduction'. For a particular poignant case consult Sean Takats, 'A Digital Humanities Tenure Case, Part 2: Letters and Committees', *The Quintessence of Ham* (7 February 2013), http://quintessenceofham.org/2013/02/07/a-digital-humanities-tenure-case-part-2-letters-and-committees/

24 Cf. also, again, van Zundert and Haentjens Dekker, 'Code, Scholarship, and Criticism'.

run versions thereof; but, in addition to this, its proper citation must become common practice in the humanities.[25] Third, reflexive accounts of (digital) humanities codework and ethnographic studies of actual work can help us understand how code and codework are changing the humanities.[26] The current contribution focuses primarily on this latter type of work by following and observing the experience of two digital humanities' programmers in order to derive insights and recommendations for those whose work may be affected by, or related to, codework in the humanities.

Methodology

The concept of the 'black box' can be seen as a methodological notion that is helpful in differentiating between the 'process' and the 'output' of knowledge production. In Latour's words, '[I]f you take two pictures, one of the 'black boxes' and the other of the open controversies, they are utterly different. They are as different as the two sides […] of a two-faced Janus. "Science in the making" on the right side, "all made science" or "ready-made science" on the other.'[27]. 'Black-boxing' can thus be perceived of as a process of enclosing the tumultuous complexity of epistemological and methodological dilemmas, controversies, compromises, and decisions that are visible in the process yet hidden in the output of knowledge production.

Our objective in this chapter is to apply Latour's first rule of method to the socio-technical context of creating an argument through codework: we will examine scholarship in the making, and follow and reopen the dilemmas and controversies of the process of knowledge production before they get enclosed in the 'black box'. We thus reopen and analyse the process of DH codework, that is, we look at the inner practices, dilemmas, and decisions of programmers as they do their work. To do this, we have used the analytical autoethnography method, which Leon

25 Juriaan H. Spaaks, 'The Research Software Directory and How It Promotes Software Citation: Improve the Findability, Citability, and Reproducibility of Research Software', *EScience Center* (11 December 2018), https://blog.esciencecenter.nl/the-research-software-directory-and-how-it-promotes-software-citation-4bd2137a6b8

26 Christine Borgman, 'The Digital Future Is Now: A Call to Action for the Humanities', *Digital Humanities Quarterly*, 3.4 (2009), www.digitalhumanities.org/dhq/vol/3/4/000077/000077.html

27 Latour, *Science in Action*, p. 4.

Anderson defines as 'ethnographic work in which the researcher is 1) a full member in the research group or setting, 2) visible as such a member in the researcher's published texts, and 3) committed to an analytic research agenda focused on improving theoretical understandings of broader social phenomena'.[28]

In our case, the analytical autoethnography unfolded in the context of a research group that consisted of three members: two DH programmers (Andrews, and Van Zundert) and one ethnographer (Antonijević). The composition of the research team enabled us to engage in a study that combined autoethnography and collaborative ethnography.[29] In practice, our study had the following methodological design:

1. The team's ethnographer formulated a set of ten questions (see Appendix 6.A) aimed at generating reflexive accounts and examples of DH codework.
2. Each of the team's DH programmers individually answered questions on a written form, providing elaborate, semi-formal accounts of his or her DH programming practice.
3. The written accounts that were generated became the basis for a series of team discussions, both written and oral, which eventually formed the 'Experiences' section of this contribution.

This method enabled us to return from the final outputs of DH codework to the scholarly uncertainties and resolutions that preceded them. Through this reconstruction we were able to document some of the key phases in the epistemological construction of code artefacts, and to identify methodologically significant moments in the stabilisation of those artefacts. In other words, we relied on the experiences of two DH hybrids: scholars proficient in both humanities research and coding, who were seeking to make explicit what DH coders themselves know, perhaps tacitly, about why and how they code. In this chapter, we have removed titles and other similar identifiers of the projects and

28 Leon Anderson, 'Analytic Autoethnography', *Journal of Contemporary Ethnography*, 35.4 (2006), 373–95, https://doi.org/10.1177/0891241605280449

29 Cf. L. E. Lassiter, *The Chicago Guide to Collaborative Ethnography*, Chicago Guides to Writing, Editing, and Publishing (Chicago, London: University of Chicago Press, 2005), http://bit.ly/2iLCmGY

software that formed the basis of our autoethnographic accounts in order to protect the privacy of the colleagues and institutions related to these projects. The written autoethnographic accounts have been quoted in their original form, except for being slightly shortened and edited for clarity.

The goal of our methodological approach was twofold: 1) enabling programmers to develop a method through which they can reflect on their practices, understand them better, and communicate them to others, and, 2) providing traditionally trained humanists with a systematic insight into the inner epistemological and methodological workings of coding in the digital humanities. As mentioned previously, we do not aspire to be representative of DH codework, but to open a debate about some of the hidden elements in this practice. Combined, these two goals could offer a better understanding of codework as an activity of knowledge production in the humanities, along with criteria for evaluating, challenging, and/or rewarding those activities. Our approach thus addressed the challenge of making codework visible again in order to understand its ontology, origin, and effects.[30]

We have grouped our observations into the categories known as 'the five canons of rhetoric' (as proposed by Cicero in his *De Inventione*): *inventio*, *dispositio*, *elocutio*, *memoria*, and *actio*. Although originally developed for public speaking, these canons have proven to be an equally potent heuristic in analysing written and, more recently, digital discourse.[31] Our contribution seeks to extend this heuristic to the analysis of coding as argumentation, not in an attempt to fit codework and its elements into a pre-defined ontology, nor to suggest that it fully conforms or matches classical rhetoric. Rather, it is a way of presenting our experiences and claims in a form that we expect will facilitate interpretation by scholars who are well-versed in text production but likely less so in codework.

All three authors have worked in professional IT contexts as part of teams that worked according to formal software development

30 Cf. Berry, *Philosophy of Software*.

31 Laura Gurak and Smiljana Antonijević, 'Digital Rhetoric and Public Discourse', in *The Sage Handbook of Rhetorical Studies*, ed. by Andrea A. Lunsford, Rosa A. Eberly, and Kirt H. Wilson (London, Thousand Oaks: SAGE Publications, Inc., 2009), pp. 497–508, https://doi.org/10.4135/9781412982795.n26

methodology (including, for example, iterative development, unit testing, code reviews, continuous integration, automatic builds and deployment, etc.). The two authors who served as subjects for the study both have dual backgrounds as formally trained academic researchers and as professional programmers. Both authors have also created and wielded bespoke code as individual programmer-researchers and as members of teams in an academic context. Their 'hybrid' skills and experience therefore make them excellent candidates to compare various types of development and academic engagement with software and source code.

Experiences

Inventio — The Impetus for DH Researchers to Code

The *inventio* stage of scholarly programming is usually driven by a specific research need: to collect data, to see a set of data in a different way, to (try to) answer a research question; to develop a new method; or to tweak some of the existing tools, resources, and/or data so as to adjust them to one's specific research needs or workflow practices. There are also other catalysts, such as being hired to do DH programming on a project, doing one's own 'free floating stuff' (as will be discussed below) and playing with technology, mastering new tools and skills, and so on. This 'spark' of invention sets off a generative process — building, tinkering, tearing down and rebuilding — that goes on until the programmer understands the parameters of the challenge.

In many cases, a humanities-specific research question will drive the software development and coding. A research design is formulated in a dialogue between developer and researcher, and it demands a workflow that can be expressed or operationalised by a developer within digital media. A particular question might be, for instance, which parts of a particular text were written by different authors: 'I searched for applicable author identification methods (which were more related to statistics than to coding) […] those methods were then "poured" into a code form for practical tests.'

Codework yields its own reflexive research questions as well, which may initiate new research and new code.

> I think [this project] is a good example. It is an attempt to find a way of coding that is closer to close reading and hermeneutics than big data analysis. [The] intent [is] to explore different modes of coding that are closer to humanities-style reasoning. For me [it is] the most intimate way of trying to find how coding is a humanities literacy.

Obviously, codework involves more than merely coding technology. Just as other types of researchers may resort to schemas, index cards, photographs, and thesauri, coding is not a single-instrument creative activity. Coders use interrogation, dialogue, drawings, and schemas in an attempt to come to a close understanding of the domain and concepts that researchers apply.

> There have been other situations where the purpose of my programming was to reverse-engineer and replicate the model of data I was given. Unfortunately the only clue I had as to the intended data model was the website that was built around the data, which means that I had to do a lot of trial-and-error guessing […] A large part of this 'programming' task was to get a big sheet of butcher paper and make an enormous diagram by hand, recording the connections between the database tables as I figured them out, and unearthing thereby the queries that were hidden on the web server. […] when I say 'hidden' I don't mean 'obscured in illegible source code' — I mean that I actually had no access to the code that contained them.

Coding can also be a means of learning and testing new skills and methods. A distinction could be made between skill-gathering projects and research projects. In research projects, the development of code will be driven by a research question, and the developer will apply well known and rehearsed tools and techniques to the problem insofar as possible. Conversely, skill gathering is driven by the need to explore and examine new tools and techniques. Such projects need not lead to actual research results, 'but coding in this sense is a good way of keeping your code skills up to date. If you're lucky enough you might draw a small paper out of that kind of coding that really is training.' In this sense coding-to-learn is equivalent to scholars keeping up with, for instance, publications in critical theory or factoring a new-found approach into an argument about the sources they are working with.

Often, method development and new research insights will co-evolve during code development:

> Perhaps the most 'scientific' programming I have done is the work on the [xyz] software. That project has been much more about trying new methods than about improving established ones, and so had a different character from the outset. I had to decide how the data ought to be collected and represented; I had to constantly revisit these decisions as I collected data that challenged my previous assumptions and heuristics. I had to discuss some concepts with computer scientists who understood more about graph arithmetic than I did in order to explain what I was trying to accomplish. The 'meat' of [this project] is a sort of calculator that, given a stemma[32] and given a set of textual variants, colors each manuscript within the stemma according to which textual variant it contains, and then works out whether that particular pattern of colors (that is to say, text mutations) could possibly have descended in a genealogical way. In a sense this is not new at all — every textual scholar understands what 'genealogical' variation implies, in the sense that they have expectations about which manuscripts in a stemma ought to share that change — but in another sense it is entirely new, since common scholarly wisdom held that there is not a lot you can do with a 'contaminated' stemma (that is, a stemma that indicates that a manuscript was copied by comparing or mixing several exemplars), but the computational model that my CS [computer science] collaborators and I developed can treat 'contaminated' stemmas in exactly the same way as traditional ones.

Thus, the argument that code begins to construct is grounded in well-established textual theory and methods. In this case, the argument is based on (parts of) the stemmatic approach (often also known as Lachmann's method), which is used to establish the genealogy of manuscripts based on variant readings that are accrued when manuscripts are copied over time. Similar to the more traditional scholarly article, the code expresses and uses these existing humanities methods and builds new methods and argument from there.

Not all code starts out with high research aspirations. Essentially, code is always used to automate work that would otherwise have been done manually, or would have been too onerous to do at all. These can be very simple tasks, such as writing a script to highlight changes between drafts of a paper, or very complex things such as writing a generic text collation tool. The development of the software is driven by tasks specific to the research at hand. But the need for coding is also born from the computational workflow itself and the need to move

32 A stemma is a tree-like representation of the genealogy of documents that represents an assertion about how later documents were copied or derived from earlier ones.

data from one data model to another. Thus, software use leads to more software needs: 'I use [this particular] web software for transcription of manuscripts, but in order to do anything with the data after I've transcribed it I need to be able to extract it from [this web software] in the form I need.'

Even though code may be geared towards facilitating tedious and repetitive, simple tasks within a research workflow, interesting and complex research designs will likely be more stimulating to developers than mundane support tasks purged of their direct relevance to the research question, for example, data preprocessing: 'Nowadays, being a senior researcher […] I will code […] when I can work from a clear research question and not from some derived coding directive.'

Related to the use of programming as a means for acquiring new methodological skills is the idea of building code as play and tinkering, an idea congruent with Geoffrey Rockwell's characterisation of text analysis and research as a form of disciplined play.[33]

> [Doing] free floating stuff. That stuff wasn't driven by research questions though. It was more solutions looking for a problem. There were these interesting text analysis methodologies and techniques, impressive statistical approaches to stylometry, etc., that just made my fingers tingle to get hands-on and to apply them to concrete problems. A friend of mine called this 'haptic thinking', a way of developing thoughts and new insights through using your keyboard.

This tinkering and play may sometimes be criticised as 'not research-driven enough', but it can actually yield very interesting results, and points to new ways of looking at a problem. However, in certain contexts, this does get recognition: for instance, some digital humanities centres give programmers a day off to pursue their own ends.[34]

Dispositio — How Coding Constructs Argument

Like text authorship, codework often consists of writing and re-writing, as well as the configuration and reconfiguration of larger pieces of code.

---

33 Geoffrey Rockwell, 'What Is Text Analysis, Really?', *Literary and Linguistic Computing*, 18.2 (2003), 209–19, https://doi.org/10.1093/llc/18.2.209

34 Smiljana Antonijević, *Amongst Digital Humanists: An Ethnographic Study of Digital Knowledge Production* (London, New York: Palgrave Macmillan, 2015), https://doi.org/10.1057/9781137484185

A programmer works by writing lines of code and combining these into larger, meaningful constructs — not unlike how lines and paragraphs of prose come into being. As with writing, much of coding's activity is to restructure individual lines of code and functions[35] until a satisfactory behaviour is achieved. This restructuring takes place at many levels: lines of code, functions, groups of such functions (called modules or libraries), and entire applications and their constituents — for example, databases, web frameworks, program core, file systems, and security layers. Many decisions are made on how to arrange these pieces while the codework is ongoing. These decisions are informed by experience, knowledge of the research domain, and considerations of feasibility, performance, and resources. They often rely as much on assumptions or educated guesses as on concrete knowledge.

Technical decisions are a necessary part of any code development trajectory. Yet, programmers learn from experience that their decisions will often change. 'A number of decisions need to be made in advance — what kind of database will I use? Is there a programming language that is particularly suitable to the task at hand, or can that decision be arbitrary? However, these decisions are essentially never final.' Technologies are swapped in and out for many reasons: performance, technical innovation, convenience of programming: '[This project] began life backed by a relational database, and then was moved to an object datastore, and is now on its way to migration into a graph database. The software was written in Perl, but its graph-database replacement is being written in Java.' Such technical decisions can affect the methodological make up of research, and it takes expertise in both coding and research design to judge them.

Thus, there is more to these decisions than purely technical considerations: a great deal depends on assumptions about and factuality of input data and research design.

> One thing that comes with experience as a programmer is the understanding that, to the extent that you do not wield perfect control over the information that is the input to your code, you are (or another developer is) probably at some point going to have to change how your code models and processes that information. This applies as much in theoretical physics or commercial software engineering as it

35 A set of the lines of code that fulfil a discrete function.

> does in the humanities; it's just that one often encounters this need for adaptation very rapidly within the humanities […] The sorts of simplistic 'shortcuts' that are common in industry or in computing in the natural sciences tend not to have a lot of useful longevity in code bases in the humanities.

This last statement in particular reveals one way in which coding in humanistic contexts tends to be distinct from coding in other domains. Humanities research deals with strongly heterogeneous data; given historical and cultural context, the importance of the situatedness of information has strong ramifications for the models and processes that are applied by the code.[36] Where the sciences may abstract away from particulars to allow patterns to emerge, the particular (the exception) is often precisely what the humanities scholar seeks. As, for instance, one of our programmers recounted: 'when you are building a prosopographical database you are not starting from formal definitions of what a "person" is and what its properties are, because "everybody knows" what a person is.' Objects and categories in the humanities are usually not as rigorously defined in their properties and attributes as are objects in the natural sciences, such as atoms or electrons. Who is an immigrant and who is native, for instance, largely depends on time, context, perspective, and who does the defining. Text is not a single stream of characters, but a complex object of layered signs and meanings, gender is far from binary, and borders of countries shift through time and geography.

Although decisions are perhaps never final, the decisions that are made can have far-reaching implications. These ramifications may occur at the level of the analytical design. For example, will a relational database be used or will a document store be applied? This choice corresponds to a primarily metadata-focused or object-focused approach. But decisions may also have institutional effects: should the software be unique bespoke code to be used only once by a single researcher, or is there an audience to be considered; and will continuous online availability have to be ensured? Such choices also lead directly to

36 Jackson, Virginia, and Lisa Gitelman, 'Introduction', in *'Raw Data' Is an Oxymoron*, ed. by Lisa Gitelman, Geoffrey C. Bowker, and Paul N. Edwards (Cambridge, MA: MIT Press, 2013), pp. 1–14, https://doi.org/10.7551/mitpress/9302.003.0002; Johanna Drucker, 'Humanities Approaches to Graphical Display', *Digital Humanities Quarterly*, 5.1 (2011), http://digitalhumanities.org/dhq/vol/5/1/000091/000091.html

decisions about life cycle management, maintenance, user support, and all the resources and management these demand.

What code will be written and how it is constructed also greatly depends on estimations of feasibility.

> If [a research design] is technically infeasible, if it is something which can't be meaningfully computed, there's no use trying. Similarly if the data is just not there or unattainable. But even if those prerequisites are met, then there's the question if it is feasible to code a solution in the time and with the resources available.

Decisions surrounding estimates of feasibility, research design, and code implementation are all comparatively informed: 'You will conjure up some of the latest on logistic regression and see if there have been similar questions, solved in similar ways, and this gives you good clues as to what and how you might do, build, and analyze.' Re-use, recombination, and reconfiguration lead to new methods and new code:

> Mostly we recycle existing ideas and we add a tiny new edge, application, or relevance to them. It is for this reason that I get suspicious if I really can't find a similar example of what I'm looking for, because 'new' mostly means a combination of what already went before but wasn't applied in a different context.

Here we see again that codework in the digital humanities follows epistemological principles that are equivalent to those in other forms of knowledge production, relying on continuous intellectual exchange with the community of practice. This is also observable in the re-writing of code. As in scholarship, argument by code is evaluated, changed, and re-evaluated in order to let it evolve into an acceptable scientific contribution. Confronted with real world data and real world use, programmers will quickly notice that many of their initial assumptions about the data, the model, and the process do not align with reality: 'Thus one gets into an iterative mode of rewriting, reworking or refactoring the code until it represents what it should represent and does what it should do.'

Both the programmers in our study feel that the most domain-relevant choices, i.e. the choices that are most pertinent to the research design and content analysis, are made in the so-called model. In codework, two types of models are usually differentiated: the data model, and the

conceptual or domain model. The first deals with the technical aspects and should ensure safe storage, interoperability, performance, and so forth. The latter model pertains to the contents, the data as meaningful concepts, and the analytical part of the research. Ideally, this model applies an idiom that mimics the concepts that are native to a (research) domain: 'it is not unrealistic to say that this conceptual model is a simulation of the research process, or the analytics in real life. In my case definitely the more relevant decisions are made in this phase. Defining, tinkering with, and exploration-wise building that model.'[37]

Even if programmers and researchers alike tend to feel that attention to the conceptual model is the most relevant part, it is generally not where most of the effort demands to be directed. As in so many fields, a tremendous amount of time is spent in data gathering and curation: 'I think a very good deal — it's like an 80/20 rule — of coding effort goes towards handling and transforming data, and usually only a lesser bit of code and coding is spent on actual analysis.'[38]

Elocutio — Coding Style, Aesthetics of Code

Software code is a thing written, and, as much as the formal constraints of computer language allow, a software author has her own style, both in regard to the aesthetics of the code and the way of working to create the code. Every coder has a personal experience of *technē* that is essential to her methods. The author may subscribe to certain methodologies, such as Agile Software Development, and she may have particular aesthetic values in coding that may be connected to such mundane things as the use of tabs instead of spaces, but these values may also relate to how program control flow and conceptual composition is used to express research domain concepts and analysis in code. Personal aesthetics can also pertain to the choices made between functionally equivalent pre-existing libraries of code and applications that are reused by the developer.

37 On modelling and its complicated relation to digital humanities work and coding, see also Willard McCarty, *Humanities Computing* (Basingstoke: Palgrave Macmillan, 2005); and Julia Flanders and Fotis Jannidis, *Knowledge Organization and Data Modeling in the Humanities* (2015), http://www.wwp.northeastern.edu/outreach/conference/kodm2012/flanders_jannidis_datamodeling.pdf

38 M. Arthur Munson, 'A Study on the Importance of and Time Spent on Different Modeling Steps', *SIGKDD Explorations*, 13.2 (2011), 65–71, https://doi.org/10.1145/2207243.2207253

A scientific discipline is often characterised by a certain dominant style of research, writing, and publication.[39] In the humanities, for instance, the monograph is generally viewed as the most valuable form of publication,[40] and an individualistic intellectual approach is preferred.[41] In a similar way there are aspects of preferred style and form to the writing of code, both individualistic and as a norm in coding communities.[42] Code writing is not a clinical process of assembling discrete mathematical logic statements; aesthetics of the code itself and a feeling of 'being in the flow' exist in code authoring just as they exist in text authoring: 'Part of the story is a feel that's somewhere in between art and craft that accompanies coding. The very feel of building, of the keyboard rattling, of code lines getting formed on the screen and those doing something. Of working your way to a working algorithm or working tool.' Yet this particular feel of flow and art in the act of building is hard to describe: 'It feels like describing the color purple.' This feel is part of the personal style of working and the personal 'poetics' of code, which is important to adhere to. Neither of the study's programmers, for example, like so-called pair programming, even though evidence exists[43] that collaborative working on code is more effective and leads to less distractions or errors, and to a boost in efficiency: 'Pair programming doesn't work for me, just like dictating doesn't work. I can't simultaneously think up complex conceptual thoughts and turn them directly and unerringly into well-formed sentences.'

Work on the interfaces for software tools intersects with the programmer's style of work and style of coding. Although neither of the programmers are very enthusiastic about interface work (see also

39 Alistair Cameron Crombie, *Styles of Scientific Thinking in the European Tradition: The History of Argument and Explanation Especially in the Mathematical and Biomedical Sciences and Arts* (London: Duckworth, 1995).

40 Peter Williams et al., 'The Role and Future of the Monograph in Arts and Humanities Research', *Aslib Proceedings*, 61.1 (2009), 67–82, https://doi.org/10.1108/00012530910932294

41 Wolfgang Kaltenbrunner, *Reflexive Inertia: Reinventing Scholarship Through Digital Practices* (Leiden: Leiden University, 2015).

42 E.g., Guido van Rossum, Barry Warsaw, and Nick Coghlan, 'PEP 8 — Style Guide for Python Code', *Python* (5 July 2001), https://www.python.org/dev/peps/pep-0008/

43 Charlie McDowell et al., 'The Impact of Pair Programming on Student Performance, Perception and Persistence', in *Proceedings of the 25th International Conference on Software Engineering*, ICSE '03 (Washington, DC: IEEE Computer Society, 2003), pp. 602–07, http://dl.acm.org/citation.cfm?id=776816.776899

the section '*Actio*') as it diverges all too soon from a research focus, it does affect the way in which the programmer uses code to develop argument. One programmer stated:

> I do like interface work as long as it is aimed at this exploring new modes of being for text, but as soon as I have to start to take care of a real user base the questions diverge from my actual research question pretty soon.

While the other programmer put it as follows:

> It does affect the way I write code, not only because I have to think a little (or a lot) harder about interface and usability when I expect others to use it, but also because I have to spend a little bit of time second-guessing how their assumptions and use cases might differ from my own. I try not to go too far in that, though — I find that engagement with real users and their needs when they actually appear is a more effective way to extend a tool than conjuring up hypothetical users and their needs.

Memoria — The Interaction between Code and Theory

We associate the rhetorical canon of *memoria* with the ability of code and codework to serve as memory systems that embed theoretical concepts within objects and recall them when needed, in order to augment research methodology and create new theory. In this way, the ability of code and codework to serve as memory systems parallels that of a book or a library. In the humanities, theory in both digital and 'conventional' fields has major and direct bearings on the programming and codework arising from these fields. We should seek, therefore, to illuminate and explain how exactly code embeds humanities theory and operates under its influence: this should not be hidden lest it be prematurely labelled a 'black box'. Similar to writing, code and coding are also an interpretation and reinterpretation of theory; like any narrative or theory, code is not some neutral re-representation: its author selects, shifts focus, expresses, and emphasises.

As methodologies go digital and their practitioners speak ever more in terms of 'data', critical theory remains fundamental to fostering understanding that there is no such thing as raw data, and that all data, including digital, is constructed, created, and situated.[44]

44 Jackson and Gitelman, 'Introduction'; Johanna Drucker, 'Humanities Approaches to Graphical Display'.

> Critical theory has a major bearing on aspects of the creation of a data model; if, for example, I were collecting a dataset in which I needed to record characteristics like 'race' or 'gender', I would have to think long and hard about how that information ought to be structured. I have run into this in the data of others, [for instance a] prosopography dataset and how it deals with the category of 'eunuchs'.

Theories from the humanities directly impact on the choice of tools and technologies that programmers use. Codework is far from theoretically uninformed, but rather theory driven.[45] One of our programmers explains that hermeneutic inference is not simply supplanted by scientific models. Rather, models evolve to express the complexity of the research object and to reflect theoretical-interpretative aspects: 'In one of our projects we started out with a very simplistic neural network that just took the vocabulary of novels as input. But to be able to correlate to readers' judgement of literary style we were soon integrating word2vec and doc2vec models to reflect theoretical notions like themes and perspective.' The other programmer explains, to illustrate, that a particular graph model used for text fits more naturally with certain arguments of post-structuralism than other models do. Transcribing a text through a model that does not assume a single, or even a single 'main', sequence of characters, is a coded reflection of an epistemological understanding that text is a multi-layered, multi-dimensional object of information, rather than a one-dimensional array of signals. When using a graph model, this programmer acknowledges making a certain set of claims about the text, and that these can be seen to be in line with post-structuralist arguments, or even with the tenets of new philology. However, this programmer also adds that such use of the model has more to do with its fitness for the research approach being tried, and less with a personal belief or conviction of what text 'should' be like: 'I am aware of that, as I use it, but at the same time I tend to want to avoid ascribing more significance to a particular computational model than it perhaps warrants.' It is important therefore to be aware of the risk of misreading the declarative nature of code: 'I think it is quite a common experience for DH programmers to have others ascribe much more

45 Jean Bauer, 'Who You Calling Untheoretical?', *Journal of Digital Humanities*, 1.1 (2011), http://journalofdigitalhumanities.org/1-1/who-you-calling-untheoretical-by-jean-bauer/

argumentative intentionality, presumption of declaration of authority, or sheer "staying power", to their models than they actually intended.' It is all too easy for programmers to be caught in this way between rival schools of thought in the humanities, and their code pointed to as evidence of a positive disregard for one critical theory or another.[46]

DH theory also adopts a reflexive stance. We may imagine, for example, a project that enters into a dialogue with the embedded ideology and discourse of big data approaches and markup languages. The discourse of the first is bound up in empiricism, quantification, scale, and speed; and thereby glosses over the precision of reasoning, the heterogeneity of data, the situatedness of data and data production, an abductive style of reasoning, and so forth, all of which are distinctive traits of many methodologies within the humanities. The discourse of big data and of markup languages is tied to certain epistemological theories: to quantification and scientism in the case of big data,[47] and to hierarchical epistemological structures and representational philosophy in the case of markup languages.[48] One of our DH programmers is engaged in a project that specifically aims to experiment with other, more hermeneutic styles of coding: 'Obviously in this case also these theories have a very direct influence in the code and coding style. I cannot, for instance, use something like machine learning unless I can convincingly argue that it serves some slow programming or close reading aspect. This is where I also still struggle very much.'

Coders take in theory and implement it. Code can be regarded as a performative application or the explanation of theory rather than a written account of it.[49] As one programmer put it:

> I guess the difference is that with code I can make it *do* something, thus I tend to try to argue through transformations of data. […] So in both print and code I somehow argue about [the object of study]. In print I do mostly by abductive logic, that is, plausible reasoning, my reasoning or my evidence is a narrative. In code I reason by transformation, I think.

46 E.g., Jones, *Emergence of the Digital Humanities*, pp. 31–32.

47 Cf. Johanna Drucker, 'Graphesis: Visual Knowledge Production and Representation', *Poetess Archive Journal*, 2.1 (2010), https://journals.tdl.org/paj/index.php/paj/article/download/4/50

48 Steven J. DeRose et al., 'What Is Text, Really?', *Journal of Computing in Higher Education*, 1.2 (1990), 3–26, https://doi.org/10.1145/264842.264847

49 Cf. Adrian Mackenzie, 'The Performativity of Code', *Theory, Culture & Society*, 22.1 (2005), 71–92, https://doi.org/10.1177/0263276405048436

An added attractiveness of code is its meticulous performance; under the same conditions code will always operate the same way, yielding the same results. This is 'an affordance that allows [one] to build [an] argument in a very iterative and controlled way'. Each statement added to a body of code is a building block of a transformative system or workflow; each bit of transformation is a bit of 'evidentiary' or 'argumentative' performance.

Lastly, codework adds to theory, as we have witnessed in an example already given above:

> In a sense this is not new at all — every textual scholar understands what 'genealogical' variation implies […] but in another sense it is entirely new, since common scholarly wisdom held that there is not a lot you can do with a 'contaminated' stemma […] but the computational model that my CS collaborators and I developed can treat 'contaminated' stemmas in exactly the same way as traditional ones.

Although it is clear that code is in some form an argument and encompasses or expresses theory, we must also acknowledge that the argumentative rhetoric of code is very limited:

> Anything that you might call an 'argument' in my code is going to be pretty oblique, or going to be a passive argument by virtue of its inclusion in my model […] academic writing tends to be expressed in rhetorical forms that are intelligible to readers, that advertise what I consider to be a fact beyond dispute, what I acknowledge as an unresolved argument but am nevertheless lending my support to one side or the other, and what I consider to be the original contribution of the argument I'm making. These rhetorical forms are entirely non-existent in code, and can only be replicated to some extent in comments and in documentation.

Actio — The Presentation and Reception of DH Codework

In codework, *actio*, the delivery of one's argument, could be compared to the publication and reception of the software and its source code. To date, DH programming is generally not recognised as a locus of humanities expertise. Coders who are trying to accumulate academic recognition and credit for their computing activities must make use of methods geared entirely toward the delivery of prose, such as publications and presentations. For scholars on the research track, most programming is done for themselves, for their own research needs, and thus with the self

as the 'intended user'. This undermines, in a meaningful and constructive way, the assumption that DH projects should always be collaborative and that their programmers should work with other humanities scholars in mind, which foregrounds the view that programmers are 'technical staff' working on behalf of researchers. Programming is often seen as a technical activity and not as research, and DH programmers are consequently seen as 'technical problem solvers' whose competencies are outside the realm of humanities expertise. This assumption is particularly entrenched for junior hybrid scholars in the 'alt-ac' (alternative-academic) careers whose professional identities and paths are often ambiguous; scholars on traditional academic tracks (especially those appointed to teach digital humanities) are accorded more recognition insofar as they have built a publication record and engaged in other forms of academic visibility and acknowledgment (conferences, projects, etc.).[50] This lends credibility to hybrid scholars, even if their traditionally trained colleagues do not fully grasp their work.

As with any other work, codework is shaped and influenced by its (social) context, which may positively or negatively influence the attitude and perception that coders hold towards their work. Both of our programmer-scholars have experienced such positive and negative influences in industry as well as in academia. One of them recalls a specific instance of a severe disconnect between management, researchers, and computer engineers. None of the groups understood much of the others' methods, motivations, commitment, or particular needs as to incentives and rewards, and were therefore unable to work very productively towards the shared research aims. This resulted in 'a lot of frustration', and a dislike, in the case of the programmer, for 'large and overcrowded' research projects. The salient point made by both programmers is that healthy interaction with others (be they co-programmers, other researchers, or management) is essential for inspired and productive research projects. Codework is very much interdisciplinary work that thrives on interaction. Both our programmers have worked in projects that had balanced and unbalanced research

50 Cf. Bethany Nowviskie, 'Where Credit Is Due: Preconditions for the Evaluation of Collaborative Digital Scholarship', *Profession* (2011), 169–81, https://doi.org/10.1632/prof.2011.2011.1.169; *#Alt-Academy 01: Alternative Academic Careers for Humanities Scholars*, ed. by Bethany Nowviskie (New York: MediaCommons Press, 2014), https://libraopen.lib.virginia.edu/public_view/6395w715k

governance; and found projects in which responsibility, accountability, and credit for design and methodology were all shared equally between humanities scholars and technologists to be far more rewarding and productive than research where all constraints were put forward by one primary investigator (PI). These observations tie in with the work of Helen Burgess and Jeanne Hamming, who argue that codework may be perceived by scholars as less of an intellectual labour than scholarly reasoning and writing.[51]

It is still hard for those doing codework in the humanities to receive acknowledgement for the academic quality or character of their software. Neither programmer reported any hint of the possibility of their code being academically evaluated or peer reviewed as digital output. Instead, academic acknowledgement and credit must be gathered through conventional venues like journals, papers, and print publication.[52] Even these garner precious little credit for the software itself, since the value of codework is often overlooked: '[T]he PIs […] almost never acknowledged in articles and presentations who did much of the work.'[53]

Some research-track programmers have managed to build a research record despite not often being acknowledged as a researcher:

> Through the years I have heard many variants of the implicit 'what you do is not research'. Someone exclaims 'But you are in IT' with a subtext of 'you're not researching'; another colleague says 'But what you do is [IT] infrastructure'; at a conference I am complimented for still being in academia as a programmer: 'You must be doing something right.'

Others have simply deployed their coding skills in the service of their own projects: 'I have been on a more classical academic tenure track. So I was never "someone else's programmer", well, not in academia anyway.'

---

51 Helen J. Burgess and Jeanne Hamming, 'New Media in Academy: Labor and the Production of Knowledge in Scholarly Multimedia', *Digital Humanities Quarterly*, 5.3 (2011), http://digitalhumanities.org/dhq/vol/5/3/000102/000102.html

52 Cf. Ryan Shaw, 'On Tenure and Why Code Can't Speak for Itself', *Ryan Shaw*, https://aeshin.org/thoughts/on-tenure/ (accessed 6 November 2017, unavailable at time of publishing).

53 Cf., for instance, James Smith, 'Coding and Digital Humanities', *James Gottlieb: Seeing What Happens When You Collide the Humanities with the Digital* (8 March 2012), http://www.jamesgottlieb.com/old/2012/03/coding-and-digital-humanities/

This inability to derive any acknowledgement for codework will eventually reflect on the work itself: 'It was very hard to derive some sense of pride and accomplishment from such projects. […] there was a consistent dissatisfaction in such projects, even though the coding, the content bit was fun.' It will eventually drive DH programmers to work primarily on projects where they are the primary investigator or have an equally visible research position: 'I will code if the coding leads to an opportunity to publish, to learn new analytic skills and methods, and when I can work from a clear research question and not from some derived coding-objective.'

When codework is not seen as genuine research contribution, it also becomes difficult for programmers to truly involve themselves in research-level discourse:

> In the earlier instances when people regarded me probably mostly as an apt programmer I only joined the research team in a phase after the general research question and design had already been discussed and set. In those cases it was not the research question that was posed to me, but […] a vague idea of 'a tool' for some purpose […] I think people generally saw me as some technical problem-solver, an implementation person, digital technician […], certainly not a researcher. [Later] researchers tended to draw me into projects earlier and started reflecting, bouncing thoughts about research questions with me.

DH programmers on a research track have little incentive to accept research support roles doing technical work on behalf of others' projects. Software development as a service does not count towards tenure and arguably contributes little to one's own research.[54] Their programming work is often bespoke code, written to meet their own needs and often not looking beyond that. This is clear when our programmers were asked about their audience. One reacted, 'It's really me. I implement the code that operationalizes my research question, or the computation-analytical part thereof', while the other stated: 'At the outset I am almost always programming for myself.'

54 Cf., again, Takats, 'Digital Humanities Tenure Case'.

Conclusions

Looking at codework from the perspective of these rhetorical canons enables us to ground this commonly overlooked research activity within the humanities framework, and to explore coding as argumentation. Our exploration showed that codework reflects humanistic discovery in that humanities-specific research questions drive coding, and tasks specific to the humanities research motivate software development. Similarly, crafting and organising code resonates with the development and arrangement of a scholarly argument, that is, a programmer writes lines of code and makes many decisions on how to arrange these pieces into larger, meaningful constructs that influence the epistemological and methodological structure of research. Our study also illustrated that, like any humanities scholar, an author of software has her own style in respect to the aesthetics of the code and in the way of working to create the code; and this style develops through both individual norms and the norms of coding communities. We also showed that, in parallel to books and libraries, code and codework serve as memory systems that embed theoretical concepts in order to augment research methodology and create new theory where code can be regarded as a performative application or explanation of theory. Finally, our ethnography has illustrated how codework *actio* compares to the publication and reception of software where DH programming is still not recognised as a locus of humanities expertise and it is hard for humanities programmers to have their code academically evaluated as digital output.

These findings illustrate that, while code and codework increasingly shape research in all fields of the humanities, they are rarely part of disciplinary discussions, remaining invisible and unknown to most scholars. This invisibility has several important consequences. First, we believe that the integration of digital scholarship into the 'humanities proper' will be at a standstill as long as methods of digital knowledge production remain mysterious, misconceived, and/or mistrusted 'carriers of alien epistemological viruses'.[55] Second, software tools become 'black boxes' in the pejorative sense, as decisions about their constitution are made, essentially, without discourse or oversight.

55 Cf. Alan Liu, 'Digital Humanities and Academic Change', *English Language Notes*, 47.1 (2009), 17–35, https://doi.org/10.1215/00138282-47.1.17

Third, 'the medium becomes the message' in the worst way: the entire being of the software and the craft that went into its making is reduced, essentially, to the user interface that is presented upon its execution — precisely the aspect that analysis oriented programmers are likely to be least concerned about — which introduces a form of 'screen essentialism'.[56] Furthermore, we have seen in the chapters above that code constructs argument, but arguably this is not clear at all for humanities scholars who do not have any coding literacy. Lastly, 'hybrid' scholars who function as DH programmers find the path to scholarly credit and recognition for their work unnecessarily difficult, and, moreover, lack avenues to systematically improve their codework through discussions with peers, exchange of best practices, or similar established methods of scholarly learning and collaboration.

Code, like text, is not self-explanatory. Software, as Joris J. van Zundert points out, contains two messages: one is received when the code is read, and the other when the code is executed.[57] Critical interrogation of the software must perforce analyse both sets of messages. Indeed, it can be too easy for scholars (whether they code or not) to verge too far toward a screen essentialism that equates the interface with the meaning of the code, or to go to the other extreme of code essentialism by asserting that code is merely another form of text to be read.

In this study we have categorised the experiences of two coders in the humanities under the familiar headings of classical rhetoric canons. This is not just a gimmick. Although Mark Marino 'would like to propose that we no longer speak of the code as a text in metaphorical terms, but that we begin to analyze and explicate code as a text, as a sign system with its own rhetoric', it remains very unclear what that rhetoric is, how it works, and what its functional elements are.[58] Following Donald Knuth, it can be argued that code is, for all practical purposes, another form of literacy, and one that is not all that alien from the literacy of text.[59] To understand its specific rhetorical character, though, we need more insights into actual coding practices. We hope we have shown that auto-

56 Matthew G. Kirschenbaum, *Mechanisms: New Media and the Forensic Imagination* (Cambridge, MA: MIT Press, 2008).

57 Van Zundert, 'Author, Editor, Engineer'.

58 Marino, 'Field Report'.

59 Donald E. Knuth, 'Literate Programming', *The Computer Journal*, 27.2 (1984), 97–111, https://doi.org/10.1093/comjnl/27.2.97

ethnographies, such as the above, play a viable role in creating such insights. Along with interviews, contextual inquiries, diaries, and other ethnographic research techniques, reflexive accounts can create a record of what actually occurs when humanists write code, thereby, yielding the information needed to understand the particular poetics, praxis, and rhetoric of code creation. We are specifically not recommending that 'everyone should learn to code'. Code nevertheless plays an ever greater, almost ubiquitous, role in culture and society; and a humanities without the capacity to critique it would be ill-prepared to investigate its own society and culture.

Recommendations

Humanities scholars will be reluctant to bring into their research anything that they feel is both beyond their capacity to understand, and insufficiently endorsed by scholars whom they trust. Therefore, a strategy is needed for making code and codework visible, understandable, trustworthy, and reputable within humanities scholarship. It must be comprehensive, both in the sense of accounting for the source code and the executed result of software, and by including all relevant stakeholders. Based on our autoethnographic observations, we provide here a set of recommendations for the various groups of stakeholders, from individual scholars to academic institutions and professional organisations. Our recommendations are in line with Stephen Ramsay and Geoffrey Rockwell's argument that the evaluation guidelines for assessing digital work usually fail to tackle the central anxiety related to DH programming, which is how to recognise and rate this work as humanistic enquiry and scholarship.[60] Where we diverge from their argument is in the focus on materialist epistemology, taken in the sense that digital artefacts should be able to communicate their underlying theoretical assumptions or claims on their own. Ramsey and Rockwell contend that such theoretical assumptions can be inferred either by using a digital artefact or through accompanying stand-in documentation. In their view, such documentation should be avoided as it reinforces the linguistic dependence of scholarly communication, diminishing the ability to communicate scholarship through artefacts. They further

60 Ramsay and Rockwell, 'Developing Things'.

argue that, although source code could be seen as 'provid[ing] an entry point to the theoretical assumptions of black boxes […] it is not at all clear that all assumptions are necessarily revealed once an application is decompiled, and few people read the code of others anyway. We are back to depending on discourse.'[61]

While we agree that providing source code is not sufficient for understanding the underlying theoretical assumptions, we disagree on viewing the 'dependence on discourse' as a feature that relativises the epistemic and communicative capacities of code and codework. In contrast, we argue that the interdependence of code and text should be embraced as a means of acknowledging their distinctive yet corresponding methods of knowledge production and communication. Just as code enhances text, making it amenable to the methodological and epistemological approaches of digital humanities; so too does text enhance code, making it more visible and intelligible for the humanities community. We believe that theoretical discussions on codework should become an established trajectory in the humanities, along with the development of methods for documenting, analysing, and evaluating code and codework. It is through the advancement of such methods and the acceptance of codework as a valid topic of humanities deliberations that the central anxiety that is related to DH programming will be cast aside. Van Zundert, following Friedrich Kittler,[62] argues that code and text literacy are on the same continuum of literacies, and are epistemologies with slightly different semiotics, which makes them well suited to reflect on each other.[63] The problems arise when either one is subordinated to the other, or when codework remains epistemologically and methodologically unexamined.

Evaluating code and DH programming in a disengaged way would be similar to the literary criticism enacted on a novel without reading it, which, in literary criticism, would be absurd. Yet it is currently the practice to 'criticise' software and code based only on the journal article that was derived from it. As much as possible, coders should support the involved evaluation of code as opposed to its disengaged criticism. On the other hand, coders regard graphical interface work as having low

61 Ibid., p. 81.

62 Van Zundert, 'Author, Editor, Engineer'; Friedrich Kittler, 'Es gibt keine Software', in *Draculas Vermächtmis* (Leipzig: Reclam Verlag, 1993), pp. 225–42.

63 Van Zundert, 'Author, Editor, Engineer'.

appeal and high risk, and that offers little true insight in the workings of the code hidden behind the interface. Both developers and users of code in the humanities should, therefore, wish to explore different forms of interfacing with code that explicitly acknowledges the interdependence of code and text. We already find ideas on such interdependence of literacies in Knuth.[64] Long dormant, these ideas have now been revived in mixed mode technologies like Jupyter Notebook,[65] which mingle code and text so that the text narrative may elucidate the code narrative. These technologies readily provide affordances for the mixed code and text interfacing that can support the involved evaluation of DH code and scholarship.

We believe that an important step in illuminating the process and results of DH programmers' codework is to develop and explicate reflexive insights into its key epistemological, methodological, and technical aspects. Explaining, for instance, what kind of research questions give impetus to one's codework and how new research insights co-evolve during code development can help both DH programmers and their traditionally trained colleagues recognise the important epistemological connections between humanistic theory and scholarly programming. As a potential starting point in developing such reflections, we provide a set of questions that guided our autoethnographic observations (Appendix 6.A). These questions are not intended to be comprehensive nor prescriptive, but rather aim to initiate a dialogue in the humanities community. In the manner of open-source software, we invite readers to explore, change, and distribute these questions in ways that best suit their research needs.

Our second set of recommendations concerns humanists who do not engage in coding. To start with, it is important to recall that first order logic (i.e. the foundation of most programming languages) has a history that starts with classical philosophy and logic, and inspired the mathematical debates that lead to the formal logic of computer languages. The philosophical roots and history of science, including humanities and computer science, from Aristotle to Bertrand Russell, demonstrate to us a productive interaction between different epistemologies, including code, embodied in the work of scholarly

64 Knuth, 'Literate Programming'.
65 Project Jupyter, *Jupyter* (2017), http://jupyter.org/

hybrids, and further advise us that we cannot relegate code to some sort of 'other' style of thought that is foreign to the humanities.

Building expertise to support digital scholarship in the humanities needs a comprehensive framework encompassing epistemological, methodological, technical, and socio-cultural aspects of digital knowledge production. These include developing an understanding of data and code, fostering critical reflection on digital objects of inquiry, and comprehending the influence of algorithmic processes on humanistic investigations. Similarly, training in digital methods should include systematic deliberation on the methodological decisions that influence research processes and results, epistemological and ethical challenges of digital scholarship, how to choose the digital tools and methods that are best suited for specific research questions, and so forth. In our view, we must reach a critical mass of humanities scholars who feel that they have the capacity to understand, if not every line of code, then at least the general thrust of what it is doing and what assumptions it is making.

When it comes to senior scholars, the results of our previous research has shown that humanists favour, and best learn, in practice, when instruction closely follows their area of study, and when it unfolds organically through collaboration with colleagues and students.[66] This is where we see rich potential for developing competencies in digital scholarship among senior humanists as well as early-career researchers — not by trying to turn them into programmers, but by tutoring them in the use of scripts and computer programs while they are engaged in their own practice of scholarship. This means showing the applicability and working of the code in a hands-on way, and qualifying them to provide informed, rather than methodologically myopic feedback on its epistemological qualities. High quality epistemological feedback is, in turn, needed to drive the development of scholarly software code forward. This requires sincere and engaged interaction between scholars who use code, and scholars who produce code: the former must be prepared to treat code and coding as more than questionably relevant; the latter must be prepared to account for their code in an academically recognisable form, such as peer review.

As previously mentioned, codework is necessarily shaped by its social context, which may positively or negatively influence the attitude

66 Cf. Antonijević, *Amongst Digital Humanists*.

and perception that both coders and other scholars hold towards their work. Our final set of recommendations thus addresses institutions and organisations, who are best placed to provide the impetus and infrastructure necessary to effect real change in how codework is received in the humanities. A necessary step in that direction is recognition and reward for peer-reviewed digital outputs, including code, as research outputs.[67] A precondition for this is to start grassroots procedures for the peer review of code,[68] and to regard code as alternative expressions of research or epistemologies with equal research value and validity, instead of subordinating code and codework to humanities proper.[69]

Another necessary step is to clarify the nature of the collaboration between coding and non-coding scholars, especially those working on the same project. Too often, DH programmers are treated as service providers instead of research focused scholars, which results in a number of negative consequences. One such consequence is that non-coding humanists appropriate all the effort and results as their own work and invention, even in cases where their only contribution was to provide a question in the form of 'can you do X…?'. Where they contribute substantial research effort and results, DH programmers should be seen as crucial peer collaborators whose competencies create a necessary link between the different areas of expertise. DH programmers have an important responsibility here too in expressing a clear appropriation of, and accountability for, their scientific programming work. Institutions can support such accountability by making it a requirement for publications that involve substantial computational analysis to have clear and comprehensive methodological descriptions of the computational approaches.

67 Cf. Nowviskie, *Where Credit is Due*; Todd Presner, 'How to Evaluate Digital Scholarship', *Journal of Digital Humanities*, 1.4 (2012), http://journalofdigitalhumanities.org/1-4/how-to-evaluate-digital-scholarship-by-todd-presner/; American Historical Association, 'Guidelines for the Professional Evaluation of Digital Scholarship by Historians', *American Historical Association* (2015), https://www.historians.org/teaching-and-learning/digital-history-resources/evaluation-of-digital-scholarship-in-history/guidelines-for-the-professional-evaluation-of-digital-scholarship-by-historians (see especially p. 1).

68 Cf. Fitzpatrick, 'Peer Review'.

69 Cf. Burgess and Hamming, 'New Media'; Ramsay and Rockwell, 'Developing Things'.

Appendix 6.A: Survey Questions

1. How does your process of DH programming usually start (e.g., with a research question you want to address; with the data you collected and need to analyse, etc.)? Give examples, if possible.

2. Who is the user you typically have in mind when programming (yourself, your team, the broader DH community)?

3. In what ways, if any, do humanities/digital humanities methods and theories influence your programming? Does this influence differ across the programming phases? Please explain and illustrate.

4. What are the main DH programming decisions you usually need to make? Do you typically think about these decisions in advance or as they appear in practice? Please explain and illustrate.

5. What would be an example of a DH argument you made through programming?

6. What are the main differences and similarities between the arguments you make in DH programming and in DH academic writing?

7. In what ways do you think humanities epistemological and methodological assumptions get reflected in your code?

8. What are the main challenges you experience in DH programming?

9. Is your DH programming typically individual or part of a collaborative project? In what ways, if any, does the decision-making differ in collaborative projects?

10. How and with whom do you usually share your code?

Bibliography

American Historical Association, 'Guidelines for the Professional Evaluation of Digital Scholarship by Historians', *American Historical Association* (2015), https://www.historians.org/teaching-and-learning/digital-history-resources/evaluation-of-digital-scholarship-in-history/guidelines-for-the-professional-evaluation-of-digital-scholarship-by-historians

Anderson, Leon, 'Analytic Autoethnography', *Journal of Contemporary Ethnography*, 35 (2006), 373–95, https://doi.org/10.1177/0891241605280449

Antonijević, Smiljana, *Amongst Digital Humanists: An Ethnographic Study of Digital Knowledge Production* (London, New York: Palgrave Macmillan, 2015), https://doi.org/10.1057/9781137484185

Bauer, Jean, 'Who You Calling Untheoretical?', *Journal of Digital Humanities*, 1 (2011), http://journalofdigitalhumanities.org/1-1/who-you-calling-untheoretical-by-jean-bauer/

Bernardi, Chiara, 'Working Towards a Definition of the Philosophy of Software', *Computational Culture*, 2 (2012), http://computationalculture.net/review/working-towards-a-definition-of-the-philosophy-of-software

Berry, David M., *Critical Theory and the Digital*, Critical Theory and Contemporary Society (New York, NY: Bloomsbury Academic, 2014), https://doi.org/10.5040/9781501302114

—— *The Philosophy of Software: Code and Mediation in the Digital Age* (Basingstoke: Palgrave Macmillan, 2011).

Bezroukov, Nikolai, 'Open Source Software Development as a Special Type of Academic Research: Critique of Vulgar Raymondism', *First Monday*, 4.10 (1999), https://doi.org/10.5210/fm.v4i10.696

Borgman, Christine, 'The Digital Future Is Now: A Call to Action for the Humanities', *Digital Humanities Quarterly*, 3.4 (2009), www.digitalhumanities.org/dhq/vol/3/4/000077/000077.html

Burgess, Helen J., and Jeanne Hamming, 'New Media in Academy: Labor and the Production of Knowledge in Scholarly Multimedia', *Digital Humanities Quarterly*, 5.3 (2011), http://digitalhumanities.org/dhq/vol/5/3/000102/000102.html

Cerquiglini, Bernard, *In Praise of the Variant: A Critical History of Philology* (Baltimore, MD: The Johns Hopkins University Press, 1999).

Coleman, E. Gabriella, *Coding Freedom: The Ethics and Aesthetics of Hacking* (Princeton (US), Woodstock (UK): Princeton University Press, 2013), http://gabriellacoleman.org/Coleman-Coding-Freedom.pdf

Coleman, G., *Hacker, Hoaxer, Whistleblower, Spy: The Many Faces of Anonymous* (London, New York: Verso, 2014).

Coyne, Richard, *Designing Information Technology in the Postmodern Age: From Method to Metaphor*, A Leonardo Book (Cambridge, MA: MIT Press, 1995).

Crombie, Alistair Cameron, *Styles of Scientific Thinking in the European Tradition: The History of Argument and Explanation Especially in the Mathematical and Biomedical Sciences and Arts* (London: Duckworth, 1995).

DeRose, Steven J., et al., 'What Is Text, Really?', *Journal of Computing in Higher Education*, 1 (1990), 3–26, https://doi.org/10.1145/264842.264847

Drucker, Johanna, 'Graphesis: Visual Knowledge Production and Representation', *Poetess Archive Journal*, 2.1 (2010), https://journals.tdl.org/paj/index.php/paj/article/download/4/50

——'Humanities Approaches to Graphical Display', *Digital Humanities Quarterly*, 5.1 (2011), http://digitalhumanities.org/dhq/vol/5/1/000091/000091.html

—— 'Should Humanists Visualize Knowledge?', *Vimeo*, video lecture at Lehigh University, Bethlehem, Pennsylvania, 2016, https://vimeo.com/140307034

Fitzpatrick, Kathleen, 'Peer Review, Judgment, and Reading', *Profession* (2011), 196–201, https://doi.org/prof.2011.2011.1.196

Flanders, Julia, and Fotis Jannidis, *Knowledge Organization and Data Modeling in the Humanities* (2015), http://www.wwp.northeastern.edu/outreach/conference/kodm2012/flanders_jannidis_datamodeling.pdf

Forsythe, D., and D. J. Hess, *Studying Those Who Study Us: An Anthropologist in the World of Artificial Intelligence* (Stanford, CA: Stanford University Press, 2001).

Galey, Alan, and Stan Ruecker, 'How a Prototype Argues', *Literary and Linguistic Computing*, 25.4 (2010), 405–24, https://doi.org/10.1093/llc/fqq021

Gurak, Laura, and Smiljana Antonijević, 'Digital Rhetoric and Public Discourse', in *The Sage Handbook of Rhetorical Studies*, ed. by Andrea A. Lunsford, Rosa A. Eberly, and Kirt H. Wilson (London, Thousand Oaks: SAGE Publications, Inc., 2017), pp. 497–508, https://doi.org/10.4135/9781412982795.n26

Hiller, Moritz, 'Signs o' the Times: The Software of Philology and a Philology of Software', *Digital Culture and Society*, 1.1 (2015), 152–63, https://doi.org/10.14361/dcs-2015-0110

Jackson, Virginia, and Lisa Gitelman, 'Introduction', in *'Raw Data' Is an Oxymoron*, ed. by Lisa Gitelman, Geoffrey C. Bowker, and Paul N. Edwards (Cambridge, MA: MIT Press, 2013), pp. 1–14, https://doi.org/10.7551/mitpress/9302.003.0002

Jones, Steven E., *The Emergence of the Digital Humanities* (New York, NY: Routledge, 2014).

Kaltenbrunner, Wolfgang, *Reflexive Inertia: Reinventing Scholarship Through Digital Practices* (Leiden: Leiden University, 2015).

Kemman, Max, Martijn Kleppe, and Stef Scagliola, 'Just Google It: Digital Research Practices of Humanities Scholars', in *Proceedings of the Digital Humanities Congress 2012*, ed. by Clare Mills, Michael Pidd, and Esther Ward (Sheffield: HRI Online Publications, 2014), arXiv:1309.2434, https://www.hrionline.ac.uk/openbook/chapter/dhc2012-kemman

Kirschenbaum, Matthew G., *Mechanisms: New Media and the Forensic Imagination* (Cambridge, MA: MIT Press, 2008).

Kittler, Friedrich, 'Es gibt keine Software', in *Draculas Vermächtmis* (Leipzig: Reclam Verlag, 1993), pp. 225–42.

Knuth, Donald E., 'Literate Programming', *The Computer Journal*, 27.2 (1984), 97–111, https://doi.org/10.1093/comjnl/27.2.97

Lassiter, L. E., *The Chicago Guide to Collaborative Ethnography*, Chicago Guides to Writing, Editing and Publishing (Chicago, London: University of Chicago Press, 2005), http://bit.ly/2iLCmGY

Latour, Bruno, *Science in Action: How to Follow Scientists and Engineers Through Society* (Cambridge, MA: Harvard University Press, 1988).

—— 'Where Are the Missing Masses, Sociology of a Few Mundane Artefacts', in *Shaping Technology-Building Society. Studies in Sociotechnical Change*, ed. by Wiebe Bijker and John Law (Cambridge, MA: MIT Press, 1992), pp. 225–59, http://www.bruno-latour.fr/node/258

Liu, Alan, 'Digital Humanities and Academic Change', *English Language Notes*, 47.1 (2009), 17–35, https://doi.org/10.1215/00138282-47.1.17

Mackenzie, Adrian, 'The Performativity of Code', *Theory, Culture & Society*, 22.1 (2005), 71–92, https://doi.org/10.1177/0263276405048436

Manovich, Lev, *Software Takes Command: Extending the Language of New Media*, International Texts in Critical Media Aesthetics 5 (New York, NY: Bloomsbury Academic, 2013).

Marino, Mark C., 'Field Report for Critical Code Studies, 2014', *Computational Culture*, 4 (2014), http://computationalculture.net/article/field-report-for-critical-code-studies-2014%E2%80%A8

McCarty, Willard, *Humanities Computing* (Basingstoke: Palgrave Macmillan, 2005).

McDowell, Charlie, et al., 'The Impact of Pair Programming on Student Performance, Perception and Persistence', in *Proceedings of the 25th International Conference on Software Engineering*, ICSE '03 (Washington, DC: IEEE Computer Society, 2003), pp. 602–07, http://dl.acm.org/citation.cfm?id=776816.776899

McPherson, Tara, 'Why Are the Digital Humanities So White? Or Thinking the Histories of Race and Computation', in *Debates in the Digital Humanities*, ed. by Matthew K. Gold (Minneapolis: University of Minnesota Press, 2012), pp.

139–60, https://doi.org/10.5749/minnesota/9780816677948.003.0017, http://dhdebates.gc.cuny.edu/debates/text/29

Munson, M. Arthur, 'A Study on the Importance of and Time Spent on Different Modeling Steps', *SIGKDD Explorations*, 13 (2011), 65–71, https://doi.org/10.1145/2207243.2207253

Nader, Laura, 'Up the Anthropologist: Perspectives Gained from Studying Up', in *Reinventing Anthropology*, ed. by D. H. Hymes, Ann Arbor Paperbacks Series (University of Michigan Press, 1972), pp. 284–311, http://www.dourish.com/classes/readings/Nader-StudyingUp.pdf

Nowviskie, Bethany, ed., *#Alt-Academy 01: Alternative Academic Careers for Humanities Scholars* (New York: MediaCommons Press, 2014), http://mediacommons.org/alt-ac/

———— 'Where Credit Is Due: Preconditions for the Evaluation of Collaborative Digital Scholarship', *Profession* (2011), 169–81, https://doi.org/10.1632/prof.2011.2011.1.169

Presner, Todd, 'How to Evaluate Digital Scholarship', *Journal of Digital Humanities*, 1.4 (2012), http://journalofdigitalhumanities.org/1-4/how-to-evaluate-digital-scholarship-by-todd-presner/

Project Jupyter, *Jupyter* (2017), http://jupyter.org/

Ramsay, Stephen, and Geoffrey Rockwell, 'Developing Things: Notes toward an Epistemology of Building in the Digital Humanities', in *Debates in the Digital Humanities*, ed. by Matthew K. Gold (Minneapolis: University of Minnesota Press, 2012), pp. 75–84, https://doi.org/10.5749/minnesota/9780816677948.003.0010, http://dhdebates.gc.cuny.edu/debates/text/11

Rockwell, Geoffrey, 'What Is Text Analysis, Really?', *Literary and Linguistic Computing*, 18.2 (2003), 209–19, https://doi.org/10.1093/llc/18.2.209

Rossum, Guido van, Barry Warsaw, and Nick Coghlan, 'PEP 8 — Style Guide for Python Code', *Python* (5 July 2001), https://www.python.org/dev/peps/pep-0008/

Schreibman, Susan, and Ann M. Hanlon, 'Determining Value for Digital Humanities Tools: Report on a Survey of Tool Developers', *Digital Humanities Quarterly*, 4.2 (2010), http://digitalhumanities.org/dhq/vol/4/2/000083/000083.html

Schreibman, Susan, Laura Mandell, and Stephen Olsen, 'Introduction', *Profession* (2011), 123–201, https://doi.org/10.1632/prof.2011.2011.1.123

Smith, James, 'Coding and Digital Humanities', *James Gottlieb: Seeing What Happens When You Collide the Humanities with the Digital* (8 March 2012), http://www.jamesgottlieb.com/old/2012/03/coding-and-digital-humanities/

Spaaks, Juriaan H., 'The Research Software Directory and How It Promotes Software Citation: Improve the Findability, Citability, and Reproducibility

of Research Software', *EScience Center* (11 December 2018), https://blog.esciencecenter.nl/the-research-software-directory-and-how-it-promotes-software-citation-4bd2137a6b8

Svensson, P., *Big Digital Humanities: Imagining a Meeting Place for the Humanities and the Digital* (Digital Culture Books, Ann Arbor, MI: University of Michigan Press, 2016), https://doi.org/10.1353/book.52252, http://hdl.handle.net/2027/spo.13607060.0001.001

Takats, Sean, 'A Digital Humanities Tenure Case, Part 2: Letters and Committees', *The Quintessence of Ham* (7 February 2013), http://quintessenceofham.org/2013/02/07/a-digital-humanities-tenure-case-part-2-letters-and-committees/

Williams, Peter, et al., 'The Role and Future of the Monograph in Arts and Humanities Research', *Aslib Proceedings*, 61.1 (2009), 67–82, https://doi.org/10.1108/00012530910932294

Zundert, Joris J. van, 'Author, Editor, Engineer: Code & the Rewriting of Authorship in Scholarly Editing', *Interdisciplinary Science Reviews*, 40.4 (2016), 349–75, https://doi.org/10.1080/03080188.2016.1165453

Zundert, Joris J. van, and Ronald Haentjens Dekker, 'Code, Scholarship, and Criticism: When Is Coding Scholarship and When Is It Not?', *Digital Scholarship in the Humanities* (2017), https://doi.org/10.1093/llc/fqx006